CommandLine

Plan für die Woche

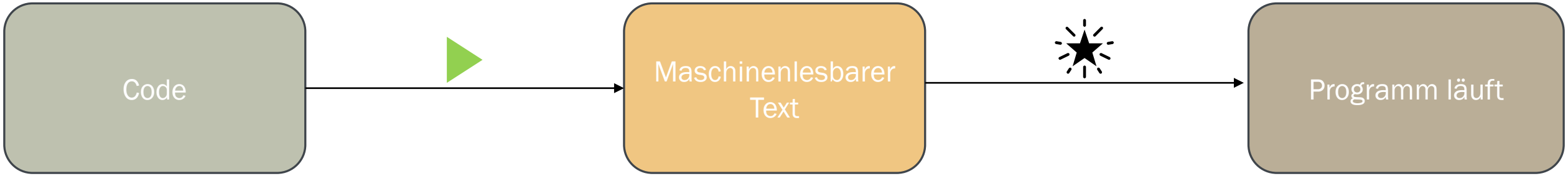
- x
- y
- z
- a
- b

Lernziele

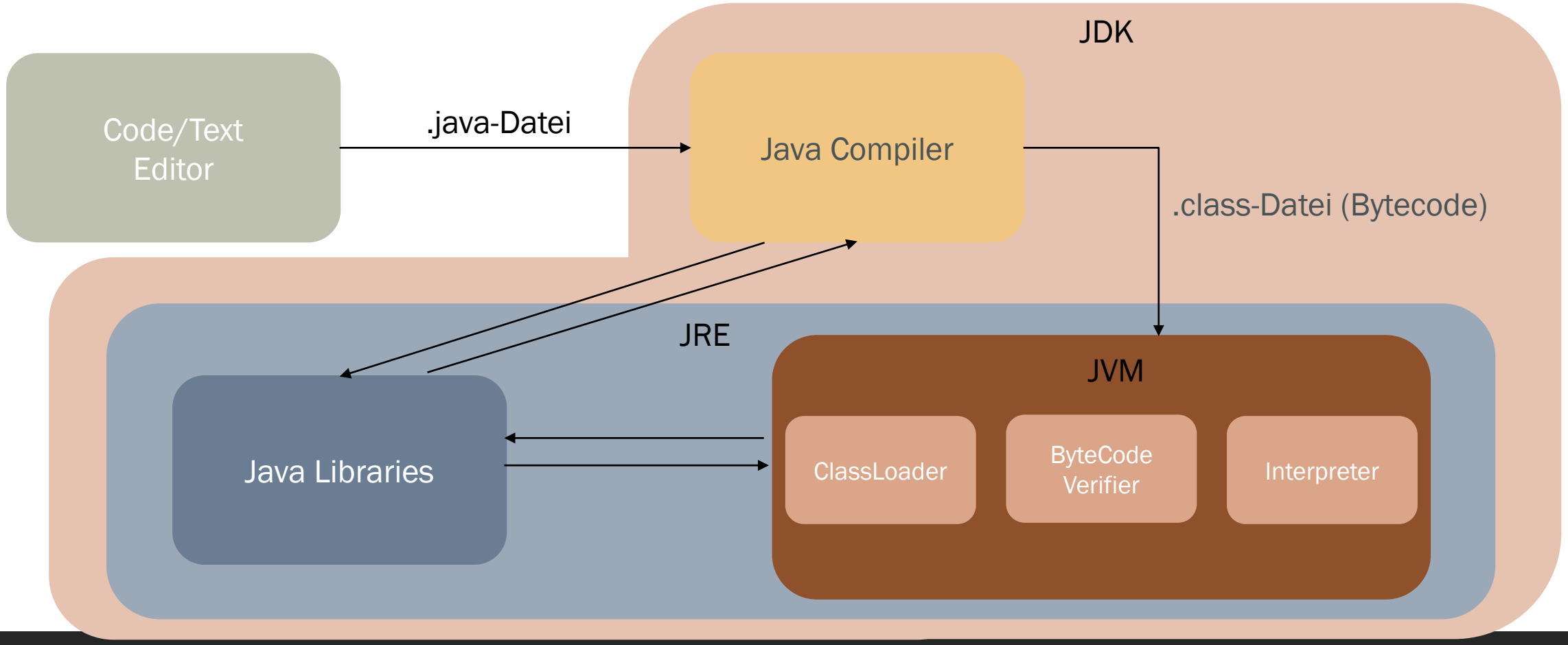
- Was ist der Unterschied zwischen JVM, JRE und JDK?
- Wichtigste Bestandteile eines JDK
- Wie arbeitet die Laufzeit (JVM)?
- Wie nutzte ich die Kommandozeile, um Java-Programme zu kompilieren und auszuführen?

Was passiert, wenn ich auf „Run“ drücke?

Lebenszyklus eines Programms

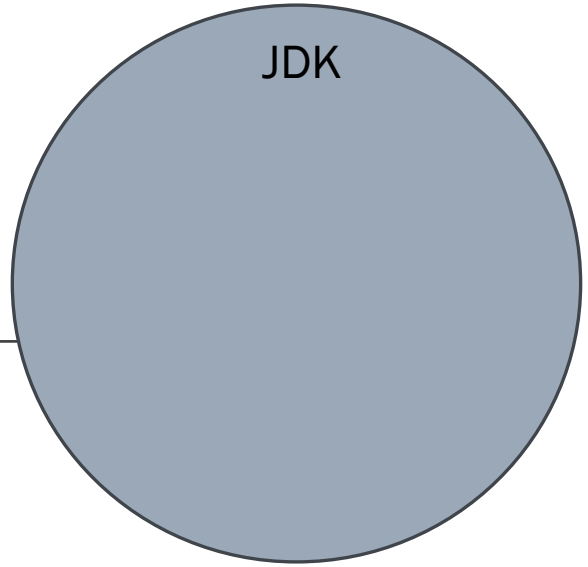


Lebenszyklus eines Programms



Aufbau einer JDK

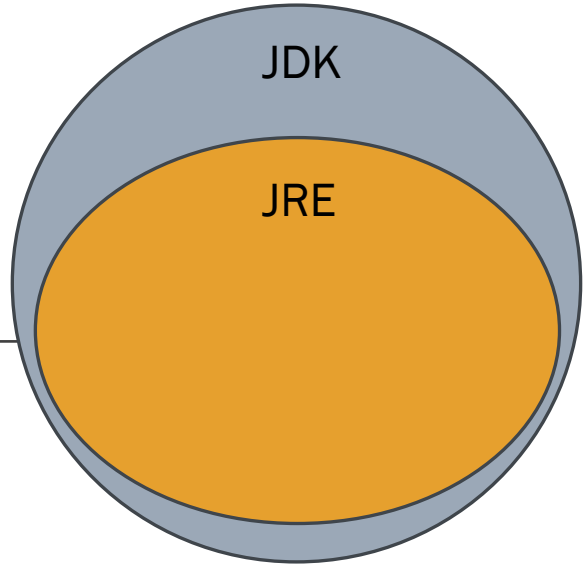
Was ist JDK?



- JDK steht für **J**ava **D**evelopment **K**it
- enthält viele Entwicklungstools (Compiler, Packwerkzeuge, Debugger,…)
- enthält die JRE
- Alles, was ein Entwickler braucht, um Java-Code zu schreiben, zu bauen und zu debuggen

Eine Werkstatt inklusive Werkzeug, um ein Auto zu bauen

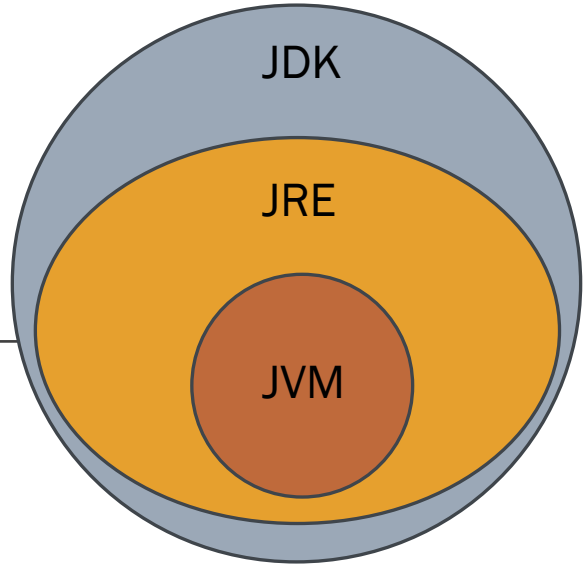
Was ist JRE?



- JRE steht für **J**ava **R**untime **E**nviroment
- ist ein Laufzeitpaket zum Ausführen von Java-Programmen
- enthält eine JVM-Implementierung und Standardbibliotheken

Motor, Tank, Elektrik -> Komponenten, um ein Auto zusammenzubauen

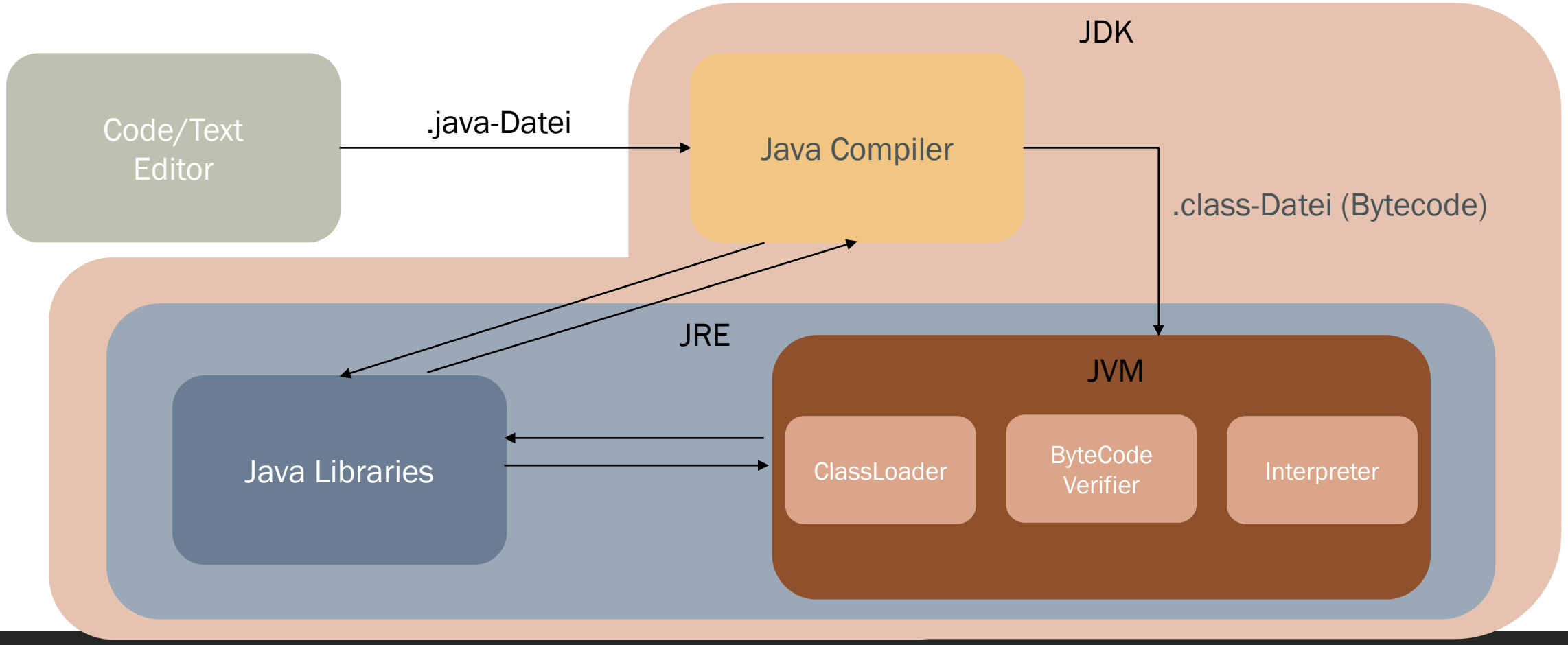
Was ist JVM?



- JVM steht für **J**ava **V**irtual **M**achine
- sie ist die virtuelle Maschine, die Java-Bytecode lädt, prüft und ausführt
- enthält Classloader, Bytecode-Interpreter, JIT-Compiler, Garbage Collector
- ohne der JVM läuft kein Java-Programm

Maschine, die das Auto mit benötigten Werkzeugen zusammenbaut

Lebenszyklus eines Programms



Quiz

Welche Komponente übersetzt den geschriebenen Code in Bytecode?

Quiz

Ist die JRE in der JDK enthalten?

Quiz

Welche Komponente ermöglicht die Plattformunabhängigkeit?

Quiz

Welche Komponente ist selbst plattformunabhängig?

Exkurs: JVM-Interna

- *ClassLoader:* lädt Klassen dynamisch in den Speicher, sobald sie benötigt werden
- *ByteCode-Verifier:* prüft den Bytecode auf Struktur,Typ- und Sicherheitsfehler
- ***J**ust-**I**n-**T**ime (JIT)-Compiler:* analysiert häufig ausgeführten Bytecode und übersetzt ihn zur Laufzeit in Maschinencode, um die Programmausführung erheblich zu beschleunigen
- *Garbage Collector: sorgt* für automatische Speicherbereinigung, indem er nicht mehr benötigte Objekte erkennt und deren Speicherplatz freigibt
- *Runtime Data Areas:* Bereiche im Speicher, die die JVM zur Programmausführung nutzt (Heap, Method Area, Stack, PC Registers)

Command Line

Kommandozeile

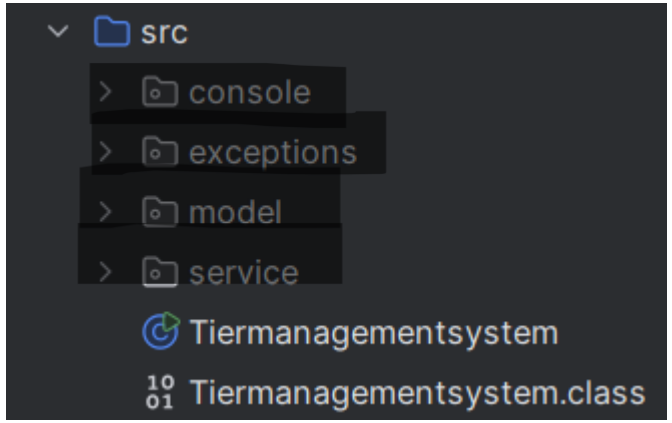
Kommandozeile

- kann zur Ausführung von Java-Programmen dienen
- Datei mittels *javac Dateiname.java* kompilieren
  - Resultat: *.class*-Datei
  - Datei wurde in *Bytecode* umgewandelt
- Ausführen der main-Methode der *.class*-Datei mittels: *java Klassenname*
- Sowohl javac als auch java können – je nach Betriebssystem – großgeschrieben werden
  - z. B. Javac/Java oder JAVAC/JAVA
  - üblich ist jedoch Kleinschreibung

Ausführen von Code

```java
public class Tiermanagementsystem {
    public static void main(String[] args) {
        System.out.println("Hello World!");
    }
}
```

```
PS C:\Users\taeOh\IdeaProjects\DemoProject> cd src
PS C:\Users\taeOh\IdeaProjects\DemoProject\src> javac Tiermanagementsystem.java
```

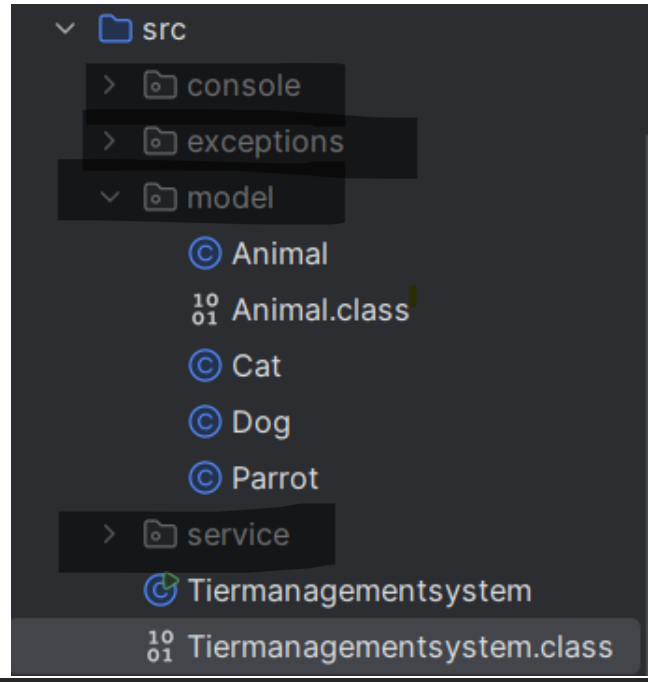


Ausführen von Code

– falls mehrere Klassen innerhalb einer Datei definiert wurden, werden sie mitkompiliert

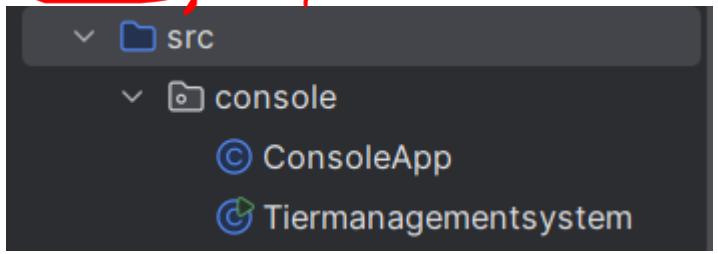
```java
import model.Animal;

public class Tiermanagementsystem {

    public static void main(String[] args) {
        Animal a = new Animal( name: "Bello",  species: "Dog",  age: 2);
        System.out.println("Hello World!");
    }
}
```

```
PS C:\Users\taeOh\IdeaProjects\DemoProject> cd src
PS C:\Users\taeOh\IdeaProjects\DemoProject\src> javac Tiermanagementsystem.java
```



Ausführen von Code in Packages

- das Ausführen von Code, welche in packages sind, ist etwas umständlicher
  - javac -d . packagePath/Dateiname.java
  - java packageName.Klassenname



-d steht für Destination und gibt an, wohin die kompilierten .class-Dateien gespeichert werden sollen

```
PS C:\Users\taeOh\IdeaProjects\DemoProject\src> javac -d . console/Tiermanagementsystem.java
PS C:\Users\taeOh\IdeaProjects\DemoProject\src> java console.Tiermanagementsystem
Hello World!
```

Übergeben von Argumenten

- Eine Main-Methode kann Argumente entgegennehmen
- diese kann man während der Ausführung in der Kommandozeile übergeben
  - Leerzeichen trennen Argumente voneinander

```java
public class Tiermanagementsystem {

    public static void main(String[] args) {
        System.out.println(args[0] + args[1]);
    }
}
```

```
PS C:\Users\taeOh\IdeaProjects\DemoProject\src> java console.Tiermanagementsystem Hallo Test
HalloTest
```

Übergeben von Argumenten

– durch das Einbetten der Argumente in „“ kann daraus ein Argument erzeugt werden

```
public class StringCommando {
    public static void main(String[] args) {
        System.out.println(args[0] + args[1]);
    }
}
```

```
PS C:\Users\Tomme\IdeaProjects\ErstesProjekt\src> javac -d.  packages/StringCommando.java
PS C:\Users\Tomme\IdeaProjects\ErstesProjekt\src> java packages.StringCommando Hallo mein Name ist Anita
Hallomein
```

```
PS C:\Users\Tomme\IdeaProjects\ErstesProjekt\src> java packages.StringCommando "Hallo mein Name ist" Anita
Hallo mein Name istAnita
```

Quiz

Womit gebe ich den Befehl, dass der geschriebene Code in Bytecode kompiliert werden soll?

Quiz

Womit gebe ich den Befehl, dass mein Programm gestartet werden soll?

Quiz

Wie kann ich Argumente übergeben? Wie kann ich mehrere Argumente in ein Argument zusammenfassen?

Exkurs: JAR-Dateien

- Java Archive (JAR) ist ein komprimiertes Archiv, das mehrere Java-Klassen, Ressourcen und Meta-Daten enthält
- im Wesentlichen: ZIP-Archiv mit einer bestimmten Struktur, das Java-Programme in einer einzigen Datei bündelt

Exkurs: JAR-Dateien

- erleichtert die Verteilung von Java-Anwendungen
  - bspw. Bibliotheken zur Verbindung mit Datenbanken (mysql-connector.jar)
- reduziert Dateigröße (wie ZIP-Dateien)

Exkurs: JAR-Dateien

- fertige Java-Programme können als .jar – Datei bereitgestellt werden
- viele Java-Bibliotheken sind als .jar-Datei veröffentlicht und können über Maven oder Gradle automatisch eingebunden werden

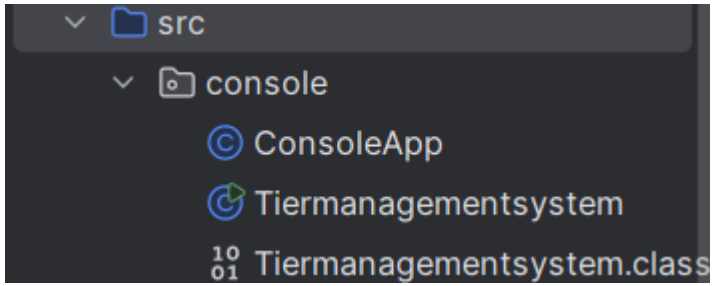
Exkurs: JAR-Dateien

- Erstellen und Ausführen einer .jar-Datei:
  - Vorher .class-Datei erzeugen
  - c -> create
  - f -> file
  - e -> entry point (Ort der main-Methode)

```
PS C:\Users\taeOh\IdeaProjects\DemoProject\src> jar cfe myFirstJar.jar Tiermanagementsystem Tiermanagementsystem.class
PS C:\Users\taeOh\IdeaProjects\DemoProject\src> java -jar myFirstJar.jar
Hello World!
```

Exkurs: JAR-Dateien

– Wenn die Dateien in einem package liegen, müssen die Pfade angepasst werden:




```
PS C:\Users\taeOh\IdeaProjects\DemoProject\src> jar cfe myFirstJar.jar console.Tiermanagementsystem console/Tiermanagementsystem.class
PS C:\Users\taeOh\IdeaProjects\DemoProject\src> jar cfe myFirstJar.jar console.Tiermanagementsystem console/Tiermanagementsystem.class
```